Credit Card Fraud Detection

using Machine Learning

Project Proposal · Machine Learning End-Semester Project

Akshita Modi · AP24110010386
Lakshay · AP24110010677
Mohana Krishna Reddy Pittu · AP24110011262
Nadimpally Dheva Harshit · AP24110011553
B.Tech Computer Science and Engineering · SRM University – AP

Supervisor
Mr. Ajay Dilip Kumar Marapatla

1

Problem Statement

Problem

What we are trying to solve

- Banks process millions of card transactions daily — a small fraction are fraudulent
- Manual review of every transaction is impossible; fraud must be flagged automatically
- The core difficulty is not classification — it is extreme class imbalance
  - Fraud is roughly 0.17% of all transactions in real data
- A model can be 99.8% accurate and still catch zero fraud
  - So accuracy is a misleading metric for this problem
- Two costs are not equal:
  - A missed fraud (false negative) costs the bank the transaction amount
  - A false alarm (false positive) costs one analyst review
- Goal: maximise fraud caught while keeping false alarms low enough to be workable

2 Abstract

Credit card fraud causes large financial losses worldwide, and the extreme rarity of fraudulent transactions makes automatic detection a demanding machine learning problem.

This project will apply the complete Machine Learning syllabus — all five units — to the detection of fraudulent transactions in a public benchmark dataset of 284,807 real transactions, of which only 492 are fraudulent.

We will train and compare a set of classifiers drawn directly from the course — decision trees, k-nearest neighbour, naive Bayes, support vector machines, perceptron and neural networks, and ensemble methods — on one common, leakage-free experimental protocol.

Because accuracy is meaningless at this imbalance, models will be judged on recall, precision, F1-score and PR-AUC. The intended outcome is a comparative study identifying which family of algorithms is best suited to rare-event detection, supported by reproducible experiments.

3 Overview

Why this problem matters

- Card fraud losses run into tens of billions of dollars a year globally
- Losses are growing as digital payments grow
- Every bank runs an automated fraud-scoring layer before human review
- The analyst team can only review a limited number of alerts per day

- Why machine learning is the right tool:
  - Fraud patterns change over time — fixed rules go stale
  - Patterns are hidden in combinations of features, not single thresholds
  - Models can be retrained as new fraud appears
- Why it is a good course project:
  - It exercises almost every algorithm family in our syllabus
  - It has a clean, public, well-studied benchmark dataset

The challenge, in one number

99.83%

is the accuracy of a model that predicts “legitimate” for every single transaction

...and it catches zero fraud.

This is why we will not report accuracy. Our models will be judged on recall, precision, F1-score and the area under the precision–recall curve.

4 Our Unique Solution

Our approach

What will make this project different

- One common experimental protocol for every model — same split, same seed, same metrics
  - Most published comparisons change the data between models, so results are not comparable
- A strict leakage-free pipeline
  - Duplicates removed before splitting; scaling fitted only on training data
  - Any resampling applied only inside cross-validation folds, never before the split
- Cost-aware decision making instead of a default 0.5 threshold
  - Using Bayes optimal decision theory (Unit III) to derive the threshold from the cost of a miss vs a false alarm
- Breadth across the whole syllabus, not one algorithm
  - Single learners, ensembles, fixed-rule and trained-rule fusion, and unsupervised clustering — compared head to head
- Full reproducibility: fixed random seed 77 (last two digits of roll number AP24110010677)

5

Dataset We Will Use

Dataset

ULB Credit Card Fraud Detection dataset (Kaggle)

- Source: Machine Learning Group, Université Libre de Bruxelles
- Real card transactions by European cardholders, September 2013
- 284,807 transactions
- 492 fraudulent — 0.172% of the data
- 31 columns, no missing values

- Features:
  - V1–V28: PCA-transformed components (anonymised for privacy)
  - Time: seconds elapsed since the first transaction
  - Amount: transaction value
  - Class: 1 = fraud, 0 = legitimate (target)
- Planned preprocessing:
  - Remove duplicate records
  - Scale Time and Amount (V1–V28 are already scaled)
  - Stratified 70 / 30 train–test split

6

Algorithms We Plan to Use

Units I–III

Drawn directly from the course syllabus

- Unit I — Foundations
  - Train / validation / test methodology
  - k-fold cross validation
  - Overfitting and underfitting analysis
- Unit II — Tree & instance-based learning
  - Decision Tree (ID3, entropy / information gain)
  - k-Nearest Neighbour
  - Feature selection: SelectKBest, mutual information, RFE
  - Dimensionality reduction: PCA, LDA
- Unit III — Probabilistic & margin learning
  - Gaussian Naive Bayes (Bayes theorem)
  - Bayes optimal decision threshold from a cost matrix
  - Probability calibration
  - Support Vector Machine (linear and RBF kernel)
  - Soft-margin / kernel trick

7 Algorithms We Plan to Use

Units IV–V

continued

- Unit IV — Neural networks
  - McCulloch–Pitts neuron and logic gates
  - Perceptron learning rule (and the XOR limitation)
  - Multilayer Perceptron trained by backpropagation
  - Gradient descent
  - Computational learning theory: VC dimension, PAC bounds
- Unit V — Ensembles & unsupervised learning
  - Bagging
  - Random Forest
  - AdaBoost
  - XGBoost (gradient boosting)
  - Fixed-rule fusion: majority / mean / product / max / min voting
  - Trained-rule fusion: stacking
  - K-means and hierarchical clustering on transaction data

8 Tools and Technologies We Plan to Use

Tools

- Language and environment
  - Python 3
  - Jupyter Notebook — one notebook per algorithm
  - Git and GitHub for version control
- Core libraries
  - NumPy, Pandas — data handling
  - scikit-learn — models, pipelines, cross validation, metrics
  - XGBoost — gradient boosting
- Visualisation
  - Matplotlib, Seaborn — confusion matrices, PR / ROC curves, trees
- Handling imbalance
  - imbalanced-learn — SMOTE and undersampling inside CV folds
- Deliverables
  - Project report (PDF)
  - Presentation
  - Public GitHub repository with all code and results

9 How We Will Do It

Method

Planned workflow

1	**Data preparation**	Load the dataset, remove duplicates, check integrity, no missing values
2	**Exploratory analysis**	Class distribution, amount and time patterns, feature correlations
3	**Common split**	Stratified 70/30 split with fixed seed 77 — reused by every model
4	**Model building**	One notebook per algorithm, each inside a scikit-learn Pipeline
5	**Validation**	5-fold stratified cross validation; resampling only inside folds
6	**Threshold tuning**	Derive the decision threshold from the cost matrix, not the default 0.5
7	**Comparison**	Rank all models on the same test set and report the findings

10

How We Will Measure Success

Evaluation

Accuracy is rejected — these are the metrics that matter

Recall	Of all frauds, how many did we catch?	Primary — a miss is the expensive error
Precision	Of all alerts raised, how many were real?	Controls analyst workload
F1-score	Harmonic mean of precision and recall	Single balanced number for ranking
PR-AUC	Area under the precision–recall curve	Best summary metric at very low prevalence
Confusion matrix	TP / FP / FN / TN counts	Shows the real cost in transactions

11 Project Goals

Goals

- Build a fraud detection system that catches a high proportion of fraudulent transactions
- Keep false alarms low enough that a small analyst team could actually review them
- Implement at least one algorithm from every unit of the syllabus
- Compare all models fairly on one identical, leakage-free protocol
- Demonstrate why accuracy fails on imbalanced data and which metrics replace it
- Derive the decision threshold from cost, using Bayes optimal decision theory
- Verify course theory on real data, not just toy examples:
  - Show the perceptron failing on XOR, as the convergence theorem predicts
  - Show the curse of dimensionality affecting k-NN as features are added
- Deliver a fully reproducible repository — fixed seed, pinned versions, all notebooks with outputs

12

Results We Are Expecting

Expected

Hypotheses we intend to test

- On model performance
  - Ensemble methods (Random Forest, XGBoost) should outperform single learners
  - Boosting expected to give the best F1-score overall
  - Naive Bayes expected to have high recall but very poor precision
  - A trivial classifier will reach ~99.83% accuracy and catch nothing
- Target operating point
  - Recall above 0.80 — catch at least 4 out of every 5 frauds
  - Precision above 0.85 — few enough false alarms to review by hand
- On theory
  - k-NN accuracy should degrade as irrelevant features are added
  - A single perceptron should fail to converge on XOR; an MLP should solve it
  - The cost-derived threshold should beat the default 0.5 threshold
- On process
  - Cross validation should confirm the test-set ranking, showing results are not a fluke
  - PCA / LDA should reduce dimensions with limited loss of detection quality

13

Work Plan and Responsibilities

Plan

Phase 1	Data preparation, EDA, common split	**Akshita Modi**
Phase 2	Units II & III models — trees, k-NN, naive Bayes, SVM, feature selection	**Mohana Krishna Reddy Pittu**
Phase 3	Unit IV — perceptron, MLP, backpropagation, learning theory	**Nadimpally Dheva Harshit**
Phase 4	Unit V — ensembles, fusion, clustering, threshold tuning	**Lakshay**
Phase 5	Comparison, report, presentation, repository	**All four members**

Thank you

Questions?

Akshita Modi

AP24110010386

Lakshay

AP24110010677

Mohana Krishna Reddy Pittu

AP24110011262

Nadimpally Dheva Harshit

AP24110011553

B.Tech Computer Science and Engineering · SRM University – AP · Supervisor: Mr. Ajay Dilip Kumar Marapatla